

20. WiFi 文字列テレメトリ bridge

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	mpc_solarcar/telemetry_text_bridge_node.py
種別	Python
区分	runtime node
役割	車両側・伴走車側から届く UDP 文字列を ROS topic に変換し、planner 指令を逆向きに文字列送信する。
起動文脈	live_wifi のセンサ入口と outbound command bridge を兼ねる。

このファイルを読む理由

車両側・伴走車側から届く UDP 文字列を ROS topic に変換し、planner 指令を逆向きに文字列送信する。

- inbound と outbound の両方向を持つ。
- speed、battery、distance、GPS、wind を ROS topic 化する。
- planner command を JSON/テキストで送り返す。

呼び出し関係

どこから呼ばれるか

- launch/solar_race_live_wifi.launch.py

次に読むと理解が進むファイル

- mpc_solarcar/telemetry_protocol.py
- mpc_solarcar/speed_command_bridge_node.py

同一リポジトリ内の主要依存

- mpc_solarcar/signal_utils.py
- mpc_solarcar/telemetry_protocol.py

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
return float(wind_speed * math.cos(rel))

class TelemetryTextBridgeNode(Node):
    def __init__(self) -> None:
        super().__init__("telemetry_text_bridge_node")
        self.declare_parameter("enable_inbound", True)
        self.declare_parameter("enable_outbound", True)
        self.declare_parameter("bind_host", "0.0.0.0")
        self.declare_parameter("bind_port", 52001)
        self.declare_parameter("publish_period_sec", 1.0)
        self.declare_parameter("solar_remote_host", "192.168.50.21")
        self.declare_parameter("solar_remote_port", 52002)
        self.declare_parameter("chase_remote_host", "192.168.50.22")
        self.declare_parameter("chase_remote_port", 52003)
        self.declare_parameter("send_to_solar", True)
        self.declare_parameter("send_to_chase", True)
        self.declare_parameter("prefer_direct_distance", False)
        self.declare_parameter("speed_filter_tau_sec", 0.6)
        self.declare_parameter("speed_max_kmh", 130.0)
        self.declare_parameter("speed_max_accel_kmhps", 12.0)
        self.declare_parameter("speed_max_decel_kmhps", 20.0)
```

主要構造の読み取り

主要クラスは TelemetryTextBridgeNode。主要関数は parse_text_payload, headwind_component_ms, main。ROS パラメータ宣言は 27 件。ROS I/O は publisher 20 件、subscription 6 件。

主要クラス・関数

行	種別	名前	補足
79	class	TelemetryTextBridgeNode	bases: Node
19	function	_safe_float	args: payload
32	function	_parse_key_value_text	args: text
53	function	parse_text_payload	args: text
66	function	headwind_component_ms	args: payload
80	function	__init__	args: self
195	function	_set_outbound	args: self, key, value
198	function	_poll_socket	args: self
238	function	_publish_gps	args: self, pub, lat, lon, alt, source_unix
253	function	_handle_vehicle_ payload	args: self, payload, source_unix
291	function	_handle_chase_payload	args: self, payload, source_unix

305	function	<code>_publish_weather</code>	args: self, payload, wind_speed, wind_dir, course_deg, headwind, now
317	function	<code>_send_payload</code>	args: self, addr, payload
325	function	<code>_send_outbound</code>	args: self
349	function	<code>main</code>	

ファイルを上から読んだときの定義順

行	内容
19	関数 <code>_safe_float</code> を定義する。
32	関数 <code>_parse_key_value_text</code> を定義する。
53	関数 <code>parse_text_payload</code> を定義する。
66	関数 <code>headwind_component_ms</code> を定義する。
79	クラス <code>TelemetryTextBridgeNode</code> を定義する。
349	関数 <code>main</code> を定義する。

import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from__future_ _importannotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>importjson</code>	manifest、checkpoint、UDP payload をやり取りするため。このファイル内での主な使用位置は L58, L321。
4	<code>importmath</code>	clip、sqrt、sin/cos、有限判定など数値ロジックに使うため。このファイル内での主な使用位置は L27, L75, L76, L180, L181, L182。
5	<code>importsocket</code>	socket モジュールを利用するため。このファイル内での主な使用位置は L166, L169。
6	<code>importtime</code>	wall/monotonic time に基づく周期制御や freshness 判定を行うため。このファイル内での主な使用位置は L216, L254, L292, L330。
7	<code>fromstatisticsimportNormalDist</code>	statistics から NormalDist を読み込み、このファイルの処理を組み立てるため。このファイル内での使用位置は少ないか、間接利用である。
9	<code>importrcclpy</code>	ROS 2 Python ノードとして起動・spin するため。このファイル内での主な使用位置は L350, L352, L354。
10	<code>fromrcclpy.nodeimportNode</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L79。
12	<code>fromsensor_msgs. msgimportNavSatFix</code>	GPS 緯度経度高度を ROS topic でやり取りするため。このファイル内での主な使用位置は L153, L154, L241。
13	<code>fromstd_msgs. msgimportFloat32,Float64, String</code>	Float32 や String など軽量メッセージで planner / vehicle 値を publish するため。このファイル内での主な使用位置は L143, L144, L145, L146, L147, L148, L149, L150, ...。

15	<code>from.signal_utilsimportRobustScalarFilter</code>	signal_utils.py から RobustScalarFilter を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/signal_utils.py。このファイル内の主な使用位置は L124, L132, L133, L134, L135, L136, L137。
16	<code>from.telemetry_protocolimportutc_iso_now, validate_source_timestamp</code>	telemetry_protocol.py から utc_iso_now, validate_source_timestamp を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/telemetry_protocol.py。このファイル内の主な使用位置は L214, L331。

関数・クラスを上から順に解説

L19 関数 _safe_float

- 定義: `_safe_float(payload, *keys)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `float, isfinite`
 - 戻り値の要点: `None / value`
1. `keys` を順に走査し、各要素を `key` に入れて処理する。
 2. `None` を返す。

L32 関数 _parse_key_value_text

- 定義: `_parse_key_value_text(text)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `float, replace, split, strip`
 - 戻り値の要点: `payload`
1. `payload` に `{}` を代入する。
 2. `text.replace(';', ',').replace('\n', ',').split(',')` を順に走査し、各要素を `token` に入れて処理する。
 3. `payload` を返す。

L53 関数 parse_text_payload

- 定義: `parse_text_payload(text)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `_parse_key_value_text, isinstance, loads, str, strip`
 - 戻り値の要点: `_parse_key_value_text(raw) / {} / parsed`
1. `raw` に `str(text or '')` の結果を代入する。
 2. 条件 `not raw` を判定し、真なら内部処理を行う。
 3. 例外処理を伴う `try` ブロックを実行する。
 4. `_parse_key_value_text(raw)` を返す。

L66 関数 headwind_component_ms

- 定義: headwind_component_ms(payload)
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: _safe_float, cos, float, radians
 - 戻り値の要点: float(wind_speed * math.cos(rel)) / direct / None
1. direct に _safe_float(payload, 'headwind_ms') の結果を代入する。
 2. 条件 direct is not None を判定し、真なら内部処理を行う。
 3. wind_speed に _safe_float(payload, 'wind_speed_ms', 'wind_ms') の結果を代入する。
 4. wind_dir に _safe_float(payload, 'wind_dir_deg') の結果を代入する。
 5. course_deg に _safe_float(payload, 'course_deg', 'heading_deg') の結果を代入する。
 6. 条件 wind_speed is None or wind_dir is None or course_deg is None を判定し、真なら内部処理を行う。
 7. rel に math.radians((wind_dir - course_deg) % 360.0) の結果を代入する。
 8. float(wind_speed * math.cos(rel)) を返す。

L79 クラス TelemetryTextBridgeNode

- 定義: TelemetryTextBridgeNode(bases=Node)
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: Float32, Float64, NavSatFix, RobustScalarFilter, String, __init__, _handle_chase_payload, _handle_vehicle_payload, _publish_gps, _publish_weather, _safe_float, _send_payload
 - 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
1. 関数 __init__ を定義する。
 2. 関数 _set_outbound を定義する。
 3. 関数 _poll_socket を定義する。
 4. 関数 _publish_gps を定義する。
 5. 関数 _handle_vehicle_payload を定義する。
 6. 関数 _handle_chase_payload を定義する。
 7. 関数 _publish_weather を定義する。
 8. 関数 _send_payload を定義する。
 9. 関数 _send_outbound を定義する。

L349 関数 main

- 定義: `main()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `TelemetryTextBridgeNode`, `destroy_node`, `init`, `shutdown`, `spin`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。

1. `rcipy.init(...)` を実行する。
2. `node` に `TelemetryTextBridgeNode()` の結果を代入する。
3. `rcipy.spin(...)` を実行する。
4. `node.destroy_node(...)` を実行する。
5. `rcipy.shutdown(...)` を実行する。

設定値と入力パラメータ

行	名前	既定値・補足
82	<code>enable_inbound</code>	<code>True</code>
83	<code>enable_outbound</code>	<code>True</code>
84	<code>bind_host</code>	<code>0.0.0.0</code>
85	<code>bind_port</code>	<code>52001</code>
86	<code>publish_period_sec</code>	<code>1.0</code>
87	<code>solar_remote_host</code>	<code>192.168.50.21</code>
88	<code>solar_remote_port</code>	<code>52002</code>
89	<code>chase_remote_host</code>	<code>192.168.50.22</code>
90	<code>chase_remote_port</code>	<code>52003</code>
91	<code>send_to_solar</code>	<code>True</code>
92	<code>send_to_chase</code>	<code>True</code>
93	<code>prefer_direct_distance</code>	<code>False</code>
94	<code>speed_filter_tau_sec</code>	<code>0.6</code>
95	<code>speed_max_kmh</code>	<code>130.0</code>
96	<code>speed_max_accel_kmhps</code>	<code>12.0</code>
97	<code>speed_max_decel_kmhps</code>	<code>20.0</code>
98	<code>distance_max_rate_kmps</code>	<code>0.06</code>
99	<code>distance_max_backtrack_km</code>	<code>0.02</code>
100	<code>battery_filter_tau_sec</code>	<code>1.0</code>
101	<code>wind_filter_tau_sec</code>	<code>1.0</code>
102	<code>headwind_filter_tau_sec</code>	<code>0.8</code>
103	<code>max_abs_headwind_ms</code>	<code>25.0</code>
104	<code>timestamp_required</code>	<code>True</code>
105	<code>max_packet_age_sec</code>	<code>5.0</code>
106	<code>max_future_skew_sec</code>	<code>2.0</code>
107	<code>max_out_of_order_sec</code>	<code>0.0</code>
108	<code>solar_power_gain_to_pack</code>	<code>1.0</code>

ROS topic I/O

行	種別	topic	callback / 補足
143	publisher	/telemetry/raw_solar	publisher
144	publisher	/telemetry/raw_chase	publisher
145	publisher	/vehicle/speed_kmh	publisher
146	publisher	/vehicle/batt_soc	publisher
147	publisher	/vehicle/batt_temp_c	publisher
148	publisher	/vehicle/batt_current_a	publisher
149	publisher	/vehicle/batt_voltage_v	publisher
150	publisher	/vehicle/solar_power_w	publisher
151	publisher	/vehicle/altitude_m	publisher
152	publisher	/vehicle/s_kmh	publisher
153	publisher	/vehicle/gps	publisher
154	publisher	/chase/gps	publisher
155	publisher	/chase/altitude_m	publisher
156	publisher	/weather/headwind_meas_ms	publisher
157	publisher	/weather/wind_speed_ms	publisher
158	publisher	/weather/wind_dir_deg	publisher
159	publisher	/weather/course_deg	publisher
160	publisher	/telemetry/bridge_status	publisher
161	publisher	/telemetry/solar_source_ts_unix	publisher
162	publisher	/telemetry/chase_source_ts_unix	publisher
184	subscription	/planner/speed_cmd	lambda msg: self._set_outbound('speed_cmd_kmh', float(msg.data))
185	subscription	/planner/upper_speed_cmd	lambda msg: self._set_outbound('upper_speed_cmd_kmh', float(msg.data))
186	subscription	/planner/drive_mode	lambda msg: self._set_outbound('drive_mode', str(msg.data))
187	subscription	/vehicle/speed_kmh	lambda msg: self._set_outbound('speed_meas_kmh', float(msg.data))
188	subscription	/vehicle/batt_soc	lambda msg: self._set_outbound('soc', float(msg.data))
189	subscription	/vehicle/s_kmh	lambda msg: self._set_outbound('s_kmh', float(msg.data))

処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。

3. timer callback 周期で主処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/telemetry_text_bridge_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。